You Asked

br.v | ctrl.v | im.v | imem.data

ir.v | pc.v | sisc.v | sisc_tb_p2-4.v

Part 2.pdf | Project overview.pdf

You are a professional in computer architecture and organization. You specialize in projects based in verilog. Attached is the project being worked on and our only goal is to fully complete part 2. There is a pair working on this project. You must have the following sections explained in the first message for part 2 and part 2 only. After the first message you will go through EVERY section and ensure not to move onto the next one until the user is ready!:

1. Explanation of the Whole Project regarding Part 2 (go through and explain what files are needing to be edited, what we are trying to accomplish, etc. all for part 2)
2. steps on how to accomplish part 2 with full marks (100% when graded). These steps will be sections in their own meaning you must go through each step as a section and not move onto the next when you get to this point. when providing the steps, you should ensure to assign person1 and person2 assignments to get this project done quickly. When doing so, please take into account that we are using github for collaboration and do not like merges at all. So, ensure that files are distributed rather than "steps" to say, if that makes sense. Make sure to list all steps initially (on the first message) so we have an idea on the length of the project. Then, you will go into each step and not move on until we are ready!
3. Tests for the user to run/do to ensure the project has been done successfully.
4. Request the user to upload only the files that were changed and are needed for submission. During this section, you must ensure to go through and do your own tests and double check that everything is looking good.

5. Give a summary of the file names, instructions on how to submit, etc. for the submission. You must ensure to go in depth when you get to this section so there's no confusion.

Gemini 3 Flash

Hello! It's great to work with you on the SISC processor. As a computer architecture specialist, I've reviewed your current files and the Part 2 requirements. You have a solid foundation from Part 1, but we have some significant "rewiring" to do to move from a manually fed instruction stream to a self-sequencing processor with branching capabilities.

Since you are using GitHub and want to avoid merge conflicts, I have structured this plan to ensure you and your partner are working on separate files as much as possible.

---

1. Project Overview: Goals for Part 2

In Part 1, your testbench likely drove the instruction register directly. In **Part 2**, we are making the processor "autonomous."

- [cite_start]**What we are accomplishing:** We are integrating the **Program Counter (PC)**, **Instruction Memory (IM)**, and **Branch (BR)** logic[cite: 81]. [cite_start]The processor will now fetch its own instructions from `imem.data` and be able to jump to different parts of the code based on branch instructions[cite: 157, 162].
- **Files to be edited:**
  - `ctrl.v`: This needs a major update. [cite_start]You must generate signals for the new modules (like `pc_write`, `pc_sel`, `ir_load`) and handle the timing for the 5-cycle instruction execution[cite: 147, 202].
  - `sisc.v`: This is currently just the ALU and Register File. [cite_start]You need to instantiate the `pc`, `im`, `ir`, and `br` modules here and "wire" them together according to the provided schematic[cite: 147].
- [cite_start]**Files to keep (unmodified):** `pc.v`, `br.v`, `ir.v`, and `im.v` are provided as-is and should not be changed[cite: 145].

---

2. Roadmap to 100% Completion

Here are the specific sections we will work through. To avoid GitHub merges, **Person 1** will primarily own the Top-Level structural file, while **Person 2** will own the Logic/FSM file.

Section A: Structural Integration (`sisc.v`)

Assigned to: Person 1

- [cite_start]Instantiate the new modules: `pc`, `im`, `ir`, and `br` [cite: 147].
- [cite_start]Wire the instruction memory to the PC and the instruction register[cite: 81].
- [cite_start]Connect the branch unit to the PC to allow for jumps[cite: 91].
- [cite_start]**Deliverable:** A completed `sisc.v` that matches the block diagram in the PDF[cite: 82].

Section B: Control Unit Logic (`ctrl.v`)

Assigned to: Person 2

- [cite_start]Implement the 5-state FSM: Fetch, Decode, Execute, Mem, Writeback[cite: 17, 18, 19].
- [cite_start]Define the logic for `IR_LOAD` (typically during Fetch)[cite: 159].
- [cite_start]Define the logic for `PC_WRITE` and `PC_SEL` to handle normal increments and branch jumps[cite: 158, 162].
- **Deliverable:** A `ctrl.v` that orchestrates the flow of data through the new hardware.

Section C: The $monitor Statement & Verification

Assigned to: Person 1 & 2 Together

- [cite_start]Update the `$monitor` in `sisc.v` to include the specific signals required for grading: `IR`, `PC`, `R1`, `R2`, `R3`, `ALU_OP`, `BR_SEL`, `PC_WRITE`, and `PC_SEL` [cite: 163, 168].

---

3. Success Tests

To ensure you get full marks, we will verify the following:

1. [cite_start]**NOP Test:** Ensure the PC increments by 1 every 5 clock cycles when reading NOPs[cite: 33, 159].
2. [cite_start]**Part 1 Regression:** Ensure the arithmetic from `imem.data` (like `ADI` and `ADD`) still results in the correct register values (R1=1, R2=8, etc.)[cite: 143, 222].
3. [cite_start]**Branch Logic:** Verify that when a `BNE` or `BRR` instruction is encountered, the `PC_SEL` and `PC_WRITE` signals trigger to jump the PC to the target address instead of just incrementing[cite: 37, 38].

---

4. Submission Prep

Once we finish the coding, I will ask you to upload your `ctrl.v` and `sisc.v`. I will review them against the block diagram to ensure no wires are crossed and the FSM timing is perfect.

---

5. Final Submission Summary

For your final ZIP file (`yourprojectname_p2.zip`), you will include:

- [cite_start]**Unchanged files:** `alu.v` , `br.v` , `im.v` , `mux32.v` , `pc.v` , `ir.v` , `rf.v` , `statreg.v` [cite: 166].
- [cite_start]**Your completed files:** `ctrl.v` and `sisc.v` [cite: 167].
- [cite_start]**Documentation:** The `work` directory and the simulation transcript[cite: 168, 169].

Are you both ready to start with Section A: Structural Integration in `sisc.v` ? (Person 1 will be the lead for the next step).

You Asked

We are ready for section a! Also, i forgot to mention, pls provide all code for any edit!

Gemini 3 Flash

Since **Person 1** is taking the lead on this section, we are focusing entirely on `sisc.v` . [cite_start]Our goal is to transform the processor from the simple ALU/RF setup of Part 1 into the full structural layout shown in your project diagram[cite: 81, 82].

Section A: Structural Integration (`sisc.v`)

Lead: Person 1

[cite_start]In this step, we are instantiating the four new modules (`pc` , `im` , `ir` , and `br`) and wiring them together[cite: 147]. According to the architecture:

- [cite_start]The **PC** (Program Counter) sends the current address to the **IM** (Instruction Memory) [cite: 54].
- [cite_start]The **IM** retrieves the 32-bit instruction and sends it to the **IR** (Instruction Register)[cite: 27, 43].
- [cite_start]The **IR** breaks that instruction into pieces (Opcode, Registers, Immediate) to feed the rest of the system[cite: 42].
- [cite_start]The **BR** (Branch) unit takes the incremented PC and the immediate value to calculate potential jump addresses[cite: 8, 160].

Updated `sisc.v` Code

Replace your current `sisc.v` content with the following. [cite_start]I have integrated the new modules and updated the `$monitor` to match the grading requirements[cite: 163, 168].

```verilog
`timescale 1ns/100ps

module sisc (clk, rst_f);
  input clk, rst_f;

  // Control Signals
  wire [3:0] alu_op;
  wire rf_we, wb_sel, br_sel, pc_rst, pc_write, pc_sel, ir_load;

  // Datapath Wires
  wire [31:0] instr;
  wire [31:0] read_data;
  wire [15:0] pc_out, br_addr;
  wire [31:0] rsa, rsb, alu_result, write_data;
  wire [3:0] stat_out, alu_stat_out, alu_stat_enable;

  // --- NEW MODULES FOR PART 2 ---

  [cite_start]// Program Counter: Holds the current instruction address [cite: 49]
  pc pc_unit (
    .clk(clk),
    .br_addr(br_addr),
    .pc_sel(pc_sel),
    .pc_write(pc_write),
    .pc_rst(pc_rst),
    .pc_out(pc_out)
  );

  [cite_start]// Instruction Memory: Stores the program [cite: 25]
  im im_unit (
    .read_addr(pc_out),
    .read_data(read_data)
  );

  [cite_start]// Instruction Register: Latches the fetched instruction [cite: 41]
  ir ir_unit (
    .clk(clk),
```

```
    .ir_load(ir_load),
    .read_data(read_data),
    .instr(instr)
  );

  [cite_start]// Branch Unit: Computes relative or absolute branch addresses
[cite: 1]
  br br_unit (
    .pc_out(pc_out),
    .imm(instr[15:0]),
    .br_sel(br_sel),
    .br_addr(br_addr)
  );

  // --- EXISTING MODULES FROM PART 1 ---

  [cite_start]// Control Unit: Updated for Part 2 [cite: 147]
  ctrl ctrl_unit (
    .clk(clk),
    .rst_f(rst_f),
    .opcode(instr[31:28]),
    .mm(instr[27:24]),
    .stat(stat_out),
    .rf_we(rf_we),
    .alu_op(alu_op),
    .wb_sel(wb_sel),
    .br_sel(br_sel),
    .pc_rst(pc_rst),
    .pc_write(pc_write),
    .pc_sel(pc_sel),
    .ir_load(ir_load)
  );

  [cite_start]// Register File [cite: 64]
  rf rf_unit (
    .clk(clk),
    .read_rega(instr[19:16]),
    .read_regb(instr[15:12]),
    .write_reg(instr[23:20]),
    .write_data(write_data),
    .rf_we(rf_we),
    .rsa(rsa),
    .rsb(rsb)
  );
```

```
  [cite_start]// ALU [cite: 65]
  alu alu_unit (
    .clk(clk),
    .rsa(rsa),
    .rsb(rsb),
    .imm(instr[15:0]),
    .alu_op(alu_op),
    .funct(instr[27:24]),
    .c_in(stat_out[3]),
    .alu_result(alu_result),
    .stat(alu_stat_out),
    .stat_en(alu_stat_enable)
  );

  [cite_start]// Write-back Mux [cite: 66]
  mux32 wb_mux (
    .in_a(alu_result),
    .in_b(32'h0),
    .sel(wb_sel),
    .out(write_data)
  );

  [cite_start]// Status Register [cite: 66]
  statreg stat_unit (
    .clk(clk),
    .in(alu_stat_out),
    .enable(alu_stat_enable),
    .out(stat_out)
  );

  // --- MONITORING ---
  [cite_start]// Updated for Part 2 requirements [cite: 163]
  initial begin
    $display("Time | IR | PC | R1 | R2 | R3 | OP | BS | PW | PS");
    $monitor("%dns | %h | %h | %h | %h | %h | %b | %b | %b | %b",
              $time, instr, pc_out,
              rf_unit.ram_array[1], rf_unit.ram_array[2], rf_unit.ram_array[3],
              alu_op, br_sel, pc_write, pc_sel);
  end

endmodule
```

[cite_start]**Note for Person 1:** I updated the port list for `ctrl_unit` to include the new signals (`br_sel` , `pc_rst` , etc.)[cite: 147]. This will technically cause a "port mismatch" error in your compiler until **Person 2** finishes the next section. This is normal when working in parallel!

Are we ready to move to Section B for Person 2 to update the Control Unit?

You Asked

let's move onto section b!

Gemini 3 Flash

This is where the "brains" of the operation come in. **Person 2**, you are now taking over `ctrl.v`.

[cite_start]In Part 2, the control unit must manage a **5-cycle FSM** (Fetch, Decode, Execute, Memory, Writeback)[cite: 14, 18, 19]. [cite_start]While Part 1 was mostly combinatorial, this version needs to handle the timing of the Program Counter and Instruction Register[cite: 157, 159].

Section B: Control Unit Logic (`ctrl.v`)

Lead: Person 2

We need to implement the following logic:

- [cite_start]**Fetch:** Assert `ir_load` to grab the instruction from memory and `pc_write` to increment the PC for the next instruction[cite: 159].
- [cite_start]**Decode:** This is where branch logic lives[cite: 162]. [cite_start]If the instruction is a branch and the condition is met, we set `pc_sel` to 1 and `pc_write` to 1 to jump[cite: 52, 53].
- [cite_start]**Execute/Mem/Writeback:** Maintain the ALU and Register File signals from Part 1[cite: 21, 23].

Updated `ctrl.v` Code

Replace your current `ctrl.v` with this implementation. I have added the branch opcode handling and the state-specific control signals.

```verilog
`timescale 1ns/100ps

module ctrl (clk, rst_f, opcode, mm, stat, rf_we, alu_op, wb_sel, br_sel,
pc_rst, pc_write, pc_sel, ir_load);
  input clk, rst_f;
  input [3:0] opcode, mm, stat;
  output reg rf_we, wb_sel, br_sel, pc_rst, pc_write, pc_sel, ir_load;
  output reg [3:0] alu_op;
```

```
  // State Definitions
  parameter fetch = 2, decode = 3, execute = 4, mem = 5, writeback = 6;

  // Opcode Definitions
  parameter NO_OP = 0, REG_OP = 1, REG_IM = 2, BNE = 6, BRR = 5, BNR = 7, HLT =
15;

  reg [2:0] present_state, next_state;

  // State Register
  always @(posedge clk or negedge rst_f) begin
    if (rst_f == 1'b0)
      present_state <= fetch;
    else
      present_state <= next_state;
  end

  // Next State Logic
  always @(*) begin
    case(present_state)
      fetch:     next_state = decode;
      decode:    next_state = execute;
      execute:   next_state = mem;
      mem:       next_state = writeback;
      writeback: next_state = fetch;
      default:   next_state = fetch;
    endcase
  end

  // Control Signal Logic
  always @(*) begin
    // Default values to avoid latches
    rf_we = 0; wb_sel = 0; alu_op = 0;
    br_sel = 0; pc_rst = 0; pc_write = 0; pc_sel = 0; ir_load = 0;

    // Reset logic
    if (rst_f == 1'b0) begin
        pc_rst = 1;
    end

    case(present_state)
      fetch: begin
        ir_load = 1;    // Load instruction from IM to IR
        pc_write = 1;   // Prepare to increment PC
        pc_sel = 0;     // Select PC + 1
```

```
      end

      decode: begin
        // Branch Logic implemented in Decode per requirements
        case (opcode)
          BNE: begin // Absolute Branch if Not Equal (Z=0)
            if (stat[0] == 1'b0) begin
              br_sel = 1;   // Absolute address
              pc_sel = 1;   // Select branch address
              pc_write = 1; // Load it into PC
            end
          end
          BRR: begin // Relative Branch (Unconditional/Always)
              br_sel = 0;   // Relative address (PC + offset)
              pc_sel = 1;   // Select branch address
              pc_write = 1; // Load it into PC
          end
          BNR: begin // Relative Branch if Not Equal (Z=0)
            if (stat[0] == 1'b0) begin
              br_sel = 0;   // Relative address
              pc_sel = 1;   // Select branch address
              pc_write = 1; // Load it into PC
            end
          end
        endcase
      end

      execute, mem, writeback: begin
        // Part 1 ALU Operations
        if (opcode == REG_OP) alu_op = 4'b0001;
        else if (opcode == REG_IM) alu_op = 4'b0011;
        else if (opcode == HLT) begin
           $display("HALT encountered at %0t", $time);
           $finish;
        end

        // Writeback logic
        if (present_state == writeback) begin
          if (opcode == REG_OP || opcode == REG_IM) rf_we = 1;
        end
      end
    endcase
  end
endmodule
```

Integration Check

[cite_start]Now that **Person 2** has finished `ctrl.v`, the port mismatch in **Person 1's** `sisc.v` should be resolved[cite: 147].

[cite_start]**Important Note on Branching:** * `BNE` uses `br_sel = 1` for an absolute jump to the immediate value[cite: 5, 37].

- [cite_start] `BRR` and `BNR` use `br_sel = 0` to add the immediate to the current `PC_out` [cite: 5, 38, 39].
- The status bit `stat[0]` typically represents the Zero (Z) flag from Part 1.

Are you ready for Section C to finalize the monitor and verify the timing?

You Asked

ok! ready for c!

Gemini 3 Flash

Now that the structural wiring (`sisc.v`) and the control logic (`ctrl.v`) are complete, we move into the verification phase. Since the project is now autonomous, we need to ensure the timing of the 5-cycle FSM aligns with the memory and register file.

Section C: The $monitor Statement & Verification

Lead: Person 1 & 2 Together

This section ensures that your simulation output provides exactly what the instructors need to grade your project. [cite_start]The `$monitor` in `sisc.v` must track the instruction execution flow through the new components[cite: 163, 168].

1. Finalizing the Monitor

[cite_start]The following block should already be in your `sisc.v` from Section A. Verify that the signals match the required list: `IR`, `PC`, `R1`, `R2`, `R3`, `ALU_OP`, `BR_SEL`, `PC_WRITE`, and `PC_SEL` [cite: 163].

```verilog

  initial begin
    $display("Time | IR | PC | R1 | R2 | R3 | OP | BS | PW | PS");
    $monitor("%dns | %h | %h | %h | %h | %h | %b | %b | %b | %b",
              $time, instr, pc_out,
```

```
                rf_unit.ram_array[1], rf_unit.ram_array[2], rf_unit.ram_array[3],
                alu_op, br_sel, pc_write, pc_sel);
end
```

2. Verification Steps (The "Success Tests")

To ensure you achieve full marks (100%), run your simulation in ModelSim and check the transcript against these criteria:

- [cite_start]**Instruction Fetch Timing:** Check that the `PC` increments every 50ns (5 cycles of 10ns each) during `NOP` or linear instructions[cite: 200, 202].
- **Part 1 Regression:** Look at the first few instructions in `imem.data`. By the time the simulation reaches the `NOP` at address `0x0B`, your registers should hold:
  - [cite_start]**R1:** `00000001` [cite: 33]
  - [cite_start]**R2:** `FF000008` [cite: 33]
  - [cite_start]**R3:** `FE000000` [cite: 33]
- **Branch Execution:** * At instruction `0x13` (`6100001D`), the `BNE` should check if `R4 != 0`. [cite_start]Since `R4` is `0` at that moment, it should **not** branch[cite: 36, 37].
  - [cite_start]At instruction `0x17` (`51000005`), the `BRR` (Relative Branch) should jump the PC forward[cite: 37, 38].
  - [cite_start]Verify that `pc_sel` and `pc_write` both go high during the **Decode** state of a successful branch[cite: 162].
- [cite_start]**Halt:** The simulation should stop when instruction `F0000000` is fetched at address `0x1D`, printing "HALT encountered"[cite: 22, 39, 40].

3. Handling GitHub Logic

Since you are both working on separate files, **Person 1** should push the finalized `sisc.v` and **Person 2** should push the finalized `ctrl.v`. Because these are different files, you should be able to pull and merge without any line conflicts.

Are you ready to move to Section 4 and upload your files for a final double-check? (Please upload `sisc.v` and `ctrl.v` once you have verified them in your simulation).